



NOMBRE DE LA ACTIVIDAD

Desarrollo Web 2 Epe1

Profesor : Sabina Solange Romero Rodríguez

Alumno : Hugo Ramirez Montano

Curso : 6101 – B1

Carrera : Ingeniera Informática

sábado, 22 de agosto de 2026

EDUCA PARA TODOS

1. Descripción del Proyecto

El proyecto "**Educa para Todos**" consiste en una plataforma web pensada para gestionar cursos de forma ordenada. El objetivo principal fue armar una aplicación web robusta utilizando tecnologías modernas como **Java 17, Jakarta EE 10 y Hibernate/JPA**, asegurando que la parte visual del sitio se comunique correctamente con la base de datos en el servidor Tomcat.

2. Cómo está estructurado (Arquitectura)

- **Modelo y Base de Datos:** Las tablas y sus relaciones (como los cursos y usuarios) se mapearon usando entidades JPA.
- **Capa de Datos (DAOs):** Cree clases DAO para manejar las consultas a la base de datos aprovechando **JPQL**, lo que me permite hacer búsquedas específicas por temas como actualizaciones masivas.
- **Interfaz de Usuario (Frontend):** 
 - **Estructura SPA:** Uso de etiquetas <section> de HTML5 para una navegación fluida en una sola página, evitando recargas constantes.
 - **Separación de responsabilidades:** Aislamiento del diseño visual (colores, tablas y tarjetas) en un archivo style.css externo, manteniendo el código HTML limpio y modular.

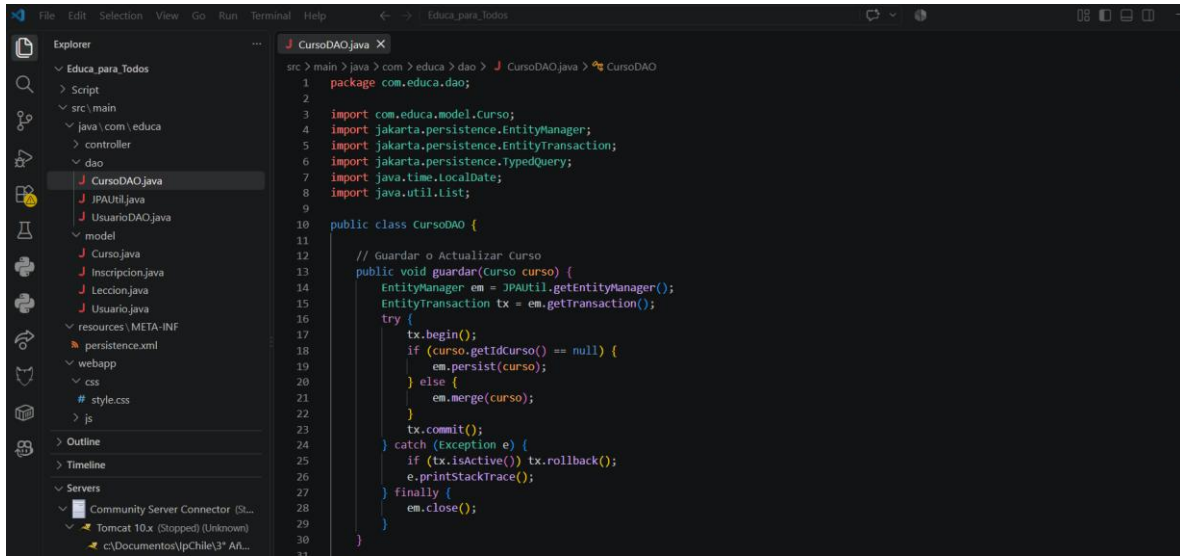
3. Mapeo Objeto-Relacional y Entidades (JPA)

o Ejemplo de Código en el Backend (Modelo Curso.java)

```

src > main > java > com > educa > model > J Curso.java > fechaCreacion
1 package com.educa.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDate;
5 import java.util.List;
6
7 @Entity
8 @Table(name = "cursos")
9 public class Curso {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "id_curso")
14     private Long idCurso;
15
16     @Column(name = "titulo", nullable = false, length = 100)
17     private String titulo;
18
19     @Column(name = "descripcion", columnDefinition = "TEXT")
20     private String descripcion;
21
22     @Column(name = "tema", length = 100)
23     private String tema;
24
25     @Column(name = "nivel_dificultad", length = 50)
26     private String nivelDificultad;
27
28     @Column(name = "fecha_creacion")
29     private LocalDate fechaCreacion;
30
31     @Column(name = "categoria", length = 50)
32     private String categoria;
33
34     @Column(name = "nivel", length = 50)
35     private String nivel;
36
37     @Column(name = "popularidad")
38     private Integer popularidad = 0;
39
40     // Campo agregado para soportar la eliminación lógica
41     @Column(name = "activo", nullable = false)
42     private Boolean activo = true;
43
44     // Relación Uno a Muchos con Leccion
45     @OneToMany(mappedBy = "curso", cascade = CascadeType.ALL, orphanRemoval = true)
46     private List<Leccion> lecciones;
47
48     public Curso() {}
49
50     public Long getIdCurso() { return idCurso; }
51     public void setIdCurso(Long idCurso) { this.idCurso = idCurso; }
52
53     public String getTitulo() { return titulo; }
54     public void setTitulo(String titulo) { this.titulo = titulo; }
55
56     public String getDescripcion() { return descripcion; }
57     public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
58
59     public String getTema() { return tema; }
60     public void setTema(String tema) { this.tema = tema; }
61
62     public String getNivelDificultad() { return nivelDificultad; }
63     public void setNivelDificultad(String nivelDificultad) { this.nivelDificultad = nivelDificultad; }
64
65     public LocalDate getFechaCreacion() { return fechaCreacion; }
66     public void setFechaCreacion(LocalDate fechaCreacion) { this.fechaCreacion = fechaCreacion; }
67
68     public String getCategoria() { return categoria; }
69     public void setCategoria(String categoria) { this.categoria = categoria; }
70
71     public String getNivel() { return nivel; }
72     public void setNivel(String nivel) { this.nivel = nivel; }
73
74     public Integer getPopularidad() { return popularidad; }
75     public void setPopularidad(Integer popularidad) { this.popularidad = popularidad; }
76
77     public Boolean getActivo() { return activo; }
78     public void setActivo(Boolean activo) { this.activo = activo; }
79
80     public List<Leccion> getLecciones() { return lecciones; }
81     public void setLecciones(List<Leccion> lecciones) { this.lecciones = lecciones; }
82 }
  
```

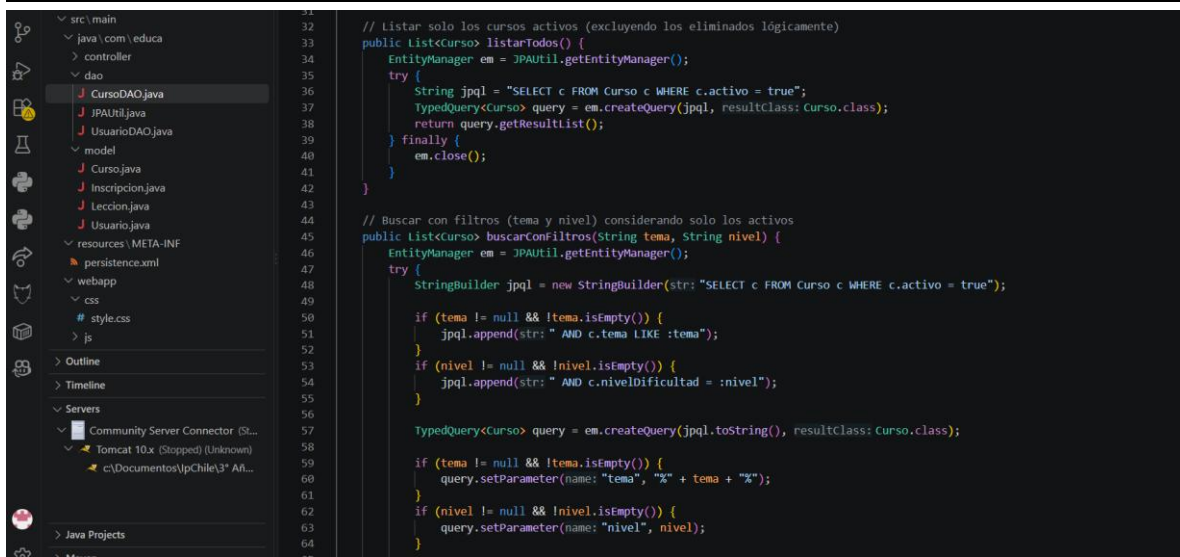
o CursoDao.java)



```

src > main > java > com > educa > dao > J CursoDAO.java > CursoDAO
1 package com.educa.dao;
2
3 import com.educa.model.curso;
4 import jakarta.persistence.EntityManager;
5 import jakarta.persistence.EntityTransaction;
6 import jakarta.persistence.TypedQuery;
7 import java.time.LocalDate;
8 import java.util.List;
9
10 public class CursoDAO {
11
12     // Guardar o Actualizar Curso
13     public void guardar(curso curso) {
14         EntityManager em = JPAUtil.getEntityManager();
15         EntityTransaction tx = em.getTransaction();
16         try {
17             tx.begin();
18             if (curso.getIdCurso() == null) {
19                 em.persist(curso);
20             } else {
21                 em.merge(curso);
22             }
23             tx.commit();
24         } catch (Exception e) {
25             if (tx.isActive()) tx.rollback();
26             e.printStackTrace();
27         } finally {
28             em.close();
29         }
30     }
31

```



```

32
33 // Listar solo los cursos activos (excluyendo los eliminados lógicamente)
34 public List<Curso> listarTodos() {
35     EntityManager em = JPAUtil.getEntityManager();
36     try {
37         String jpql = "SELECT c FROM Curso c WHERE c.activo = true";
38         TypedQuery<Curso> query = em.createQuery(jpql, resultadoClass: Curso.class);
39         return query.getResultList();
40     } finally {
41         em.close();
42     }
43 }
44
45 // Buscar con filtros (tema y nivel) considerando solo los activos
46 public List<Curso> buscarConFiltros(String tema, String nivel) {
47     EntityManager em = JPAUtil.getEntityManager();
48     try {
49         StringBuilder jpql = new StringBuilder(str: "SELECT c FROM Curso c WHERE c.activo = true");
50
51         if (tema != null && !tema.isEmpty()) {
52             jpql.append(str: " AND c.tema LIKE :tema");
53         }
54         if (nivel != null && !nivel.isEmpty()) {
55             jpql.append(str: " AND c.nivelDificultad = :nivel");
56         }
57
58         TypedQuery<Curso> query = em.createQuery(jpql.toString(), resultadoClass: Curso.class);
59
60         if (tema != null && !tema.isEmpty()) {
61             query.setParameter(name: "tema", "%" + tema + "%");
62         }
63         if (nivel != null && !nivel.isEmpty()) {
64             query.setParameter(name: "nivel", nivel);
65         }
66     } finally {
67         em.close();
68     }
69 }
70

```

```

66         return query.getResultList();
67     } finally {
68         em.close();
69     }
70 }
71
72 // Operación Masiva: Actualizar popularidad según umbral
73 public int actualizarPopularidadMasiva(int umbral, int nuevaPopularidad) {
74     EntityManager em = JPAUtil.getEntityManager();
75     EntityTransaction tx = em.getTransaction();
76     try {
77         tx.begin();
78         String jpql = "UPDATE Curso c SET c.popularidad = :nuevaPop WHERE c.popularidad < :umbral AND c.activo = true";
79         int filasAfectadas = em.createQuery(jpql)
80             .setParameter(name: "nuevaPop", nuevaPopularidad)
81             .setParameter(name: "umbral", umbral)
82             .executeUpdate();
83         tx.commit();
84         return filasAfectadas;
85     } catch (Exception e) {
86         if (tx.isActive()) tx.rollback();
87         e.printStackTrace();
88         return 0;
89     } finally {
90         em.close();
91     }
92 }
93
94 // Operación Masiva: ELIMINACIÓN LÓGICA (Cambia activo a false)
95 public int eliminarCursosBajaPopularidad(int limite) {
96     EntityManager em = JPAUtil.getEntityManager();
97     EntityTransaction tx = em.getTransaction();
98     try {
99         tx.begin();
100         String jpql = "UPDATE Curso c SET c.activo = false WHERE c.popularidad < :limite AND c.activo = true";
101         int filasAfectadas = em.createQuery(jpql)
102             .setParameter(name: "limite", limite)
103             .executeUpdate();
104         tx.commit();
105         return filasAfectadas;
106     } catch (Exception e) {
107         if (tx.isActive()) tx.rollback();
108         e.printStackTrace();
109         return 0;
110     } finally {
111         em.close();
112     }
113 }
114
115 // Operación Masiva: Incrementar popularidad para cursos recientes (nombre exacto requerido por el servlet)
116 public int incrementarPopularidadCursosRecientes(LocalDate fechaLimite, int incremento) {
117     EntityManager em = JPAUtil.getEntityManager();
118     EntityTransaction tx = em.getTransaction();
119     try {
120         tx.begin();
121         String jpql = "UPDATE Curso c SET c.popularidad = c.popularidad + :inc WHERE c.fechaCreacion >= :fecha AND c.activo = true";
122         int filas = em.createQuery(jpql)
123             .setParameter(name: "inc", incremento)
124             .setParameter(name: "fecha", fechaLimite)
125             .executeUpdate();
126         tx.commit();
127         return filas;
128     } catch (Exception e) {
129         if (tx.isActive()) tx.rollback();
130         e.printStackTrace();
131         return 0;
132     } finally {
133         em.close();
134     }
135 }
136 }

```

Entidad Usuario.java

```

1 package com.educa.model;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.JoinColumn;
9 import jakarta.persistence.JoinTable;
10 import jakarta.persistence.ManyToMany;
11 import jakarta.persistence.Table;
12 import java.io.Serializable;
13 import java.time.LocalDate;
14 import java.util.List;
15

```

```

16 @Entity
17 @Table(name = "usuarios")
18 public class Usuario implements Serializable {
19
20     private static final long serialVersionUID = 1L;
21
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     @Column(name = "id_usuario")
25     private Long idUsuario;
26
27     @Column(nullable = false, length = 100)
28     private String nombre;
29
30     @Column(nullable = false, unique = true, length = 100)
31     private String email;
32
33     @Column(nullable = false, length = 100)
34     private String password;
35
36     @Column(nullable = false, length = 50)
37     private String rol; // Ej: Estudiante, Administrador
38

```

```

38
39     @Column(length = 30)
40     private String estado;
41
42     @Column(name = "fecha_registro")
43     private LocalDate fechaRegistro;
44
45     // Relación: Un usuario puede inscribirse en muchos cursos
46     @ManyToMany
47     @JoinTable(
48         name = "inscripciones",
49         joinColumns = @JoinColumn(name = "id_usuario"),
50         inverseJoinColumns = @JoinColumn(name = "id_curso")
51     )
52     private List<Curso> cursosInscritos;
53
54     public Usuario() {}
55

```

```

55
56
57 public Usuario(String nombre, String email, String password, String rol) {
58     this.nombre = nombre;
59     this.email = email;
60     this.password = password;
61     this.rol = rol;
62     this.fechaRegistro = LocalDate.now();
63 }
64
65 // Getters y Setters
66 public Long getIdUsuario() { return idUsuario; }
67 public void setIdUsuario(Long idUsuario) { this.idUsuario = idUsuario; }
68
69 public String getNombre() { return nombre; }
70 public void setNombre(String nombre) { this.nombre = nombre; }
71
72 public String getEmail() { return email; }
73 public void setEmail(String email) { this.email = email; }
74
75 public String getPassword() { return password; }
76 public void setPassword(String password) { this.password = password; }
77
78 public void setRol(String rol) { this.rol = rol; }
79
80 public String getEstado() { return estado; }
81 public void setEstado(String estado) { this.estado = estado; }
82
83 public LocalDate getFechaRegistro() { return fechaRegistro; }
84 public void setFechaRegistro(LocalDate fechaRegistro) { this.fechaRegistro = fechaRegistro; }
85
86 public List<Curso> getCursosInscritos() { return cursosInscritos; }
87 public void setCursosInscritos(List<Curso> cursosInscritos) { this.cursosInscritos = cursosInscritos; }
88 }

```

o Ejemplo de Dao UsuarioDao.java)

```

src > main > java > com > educa > dao > J UsuarioDAO.java > % UsuarioDAO

1  package com.educa.dao;
2
3  import com.educa.model.Usuario;
4  import jakarta.persistence.EntityManager;
5  import jakarta.persistence.TypedQuery;
6  import java.time.LocalDate;
7  import java.util.List;
8
9  public class UsuarioDAO {
10
11     // 1. Listar todos los usuarios
12     public List<Usuario> listarTodos() {
13         EntityManager em = JPAUtil.getEntityManager();
14         try {
15             String jpql = "SELECT u FROM Usuario u";
16             TypedQuery<Usuario> query = em.createQuery(jpql, Usuario.class);
17             return query.getResultList();
18         } finally {
19             em.close();
20         }
21     }
22
23     // 2. Buscar usuario por ID
24     public Usuario buscarPorId(Long id) {
25         EntityManager em = JPAUtil.getEntityManager();
26         try {
27             return em.find(entityClass: Usuario.class, id);
28         } finally {
29             em.close();
30         }
31     }
32
33     // 3. Guardar o actualizar un usuario (CRUD)
34     public void guardar(Usuario usuario) {
35         EntityManager em = JPAUtil.getEntityManager();
36         try {
37             em.getTransaction().begin();
38             if (usuario.getIdUsuario() == null) {
39                 em.persist(usuario);
40             } else {
41                 em.merge(usuario);
42             }
43             em.getTransaction().commit();
44         } catch (Exception e) {
45             if (em.getTransaction().isActive()) {
46                 em.getTransaction().rollback();
47             }
48             throw e;
49         } finally {
50             em.close();
51         }
52     }
53
54     // 4. Eliminar un usuario por ID
55     public void eliminar(Long id) {
56         EntityManager em = JPAUtil.getEntityManager();
57         try {
58             em.getTransaction().begin();
59             Usuario usuario = em.find(entityClass: Usuario.class, id);
60             if (usuario != null) {
61                 em.remove(usuario);
62             }
63             em.getTransaction().commit();
64         } catch (Exception e) {
65             if (em.getTransaction().isActive()) {
66                 em.getTransaction().rollback();
67             }
68             throw e;
69         } finally {
70             em.close();
71         }
72     }
73 }

```

```

73
74 // 5. Operación UPDATE Masiva requerida por el Servlet (Borrado lógico / desactivación)
75 public int borradoLogicoUsuariosMasivo(LocalDate fechaLimite) {
76     EntityManager em = JPAUtil.getEntityManager();
77     try {
78         em.getTransaction().begin();
79         String jpql = "UPDATE Usuario u SET u.estado = 'INACTIVO' WHERE u.fechaRegistro < :fecha AND u.estado = 'ACTIVO'";
80         int filasAfectadas = em.createQuery(jpql)
81             .setParameter(name: "fecha", fechaLimite)
82             .executeUpdate();
83         em.getTransaction().commit();
84         return filasAfectadas;
85     } catch (Exception e) {
86         if (em.getTransaction().isActive()) {
87             em.getTransaction().rollback();
88         }
89         throw e;
90     } finally {
91         em.close();
92     }
93 }
94
95 // 6. Operación DELETE Masiva: Eliminar usuarios inactivos
96 public int eliminarUsuariosInactivos() {
97     EntityManager em = JPAUtil.getEntityManager();
98     try {
99         em.getTransaction().begin();
100         String jpql = "DELETE FROM Usuario u WHERE u.estado = 'INACTIVO'";
101         int filasAfectadas = em.createQuery(jpql).executeUpdate();
102         em.getTransaction().commit();
103         return filasAfectadas;
104     } catch (Exception e) {
105         if (em.getTransaction().isActive()) {
106             em.getTransaction().rollback();
107         }
108         throw e;
109     } finally {
110         em.close();
111     }
112 }
113 }

```

Explicación técnica del Mapeo Relacional (@ManyToOne y @OneToMany)

Para cumplir con los requisitos del modelo relacional, estructuré la conexión entre los cursos y sus contenidos de la siguiente manera:

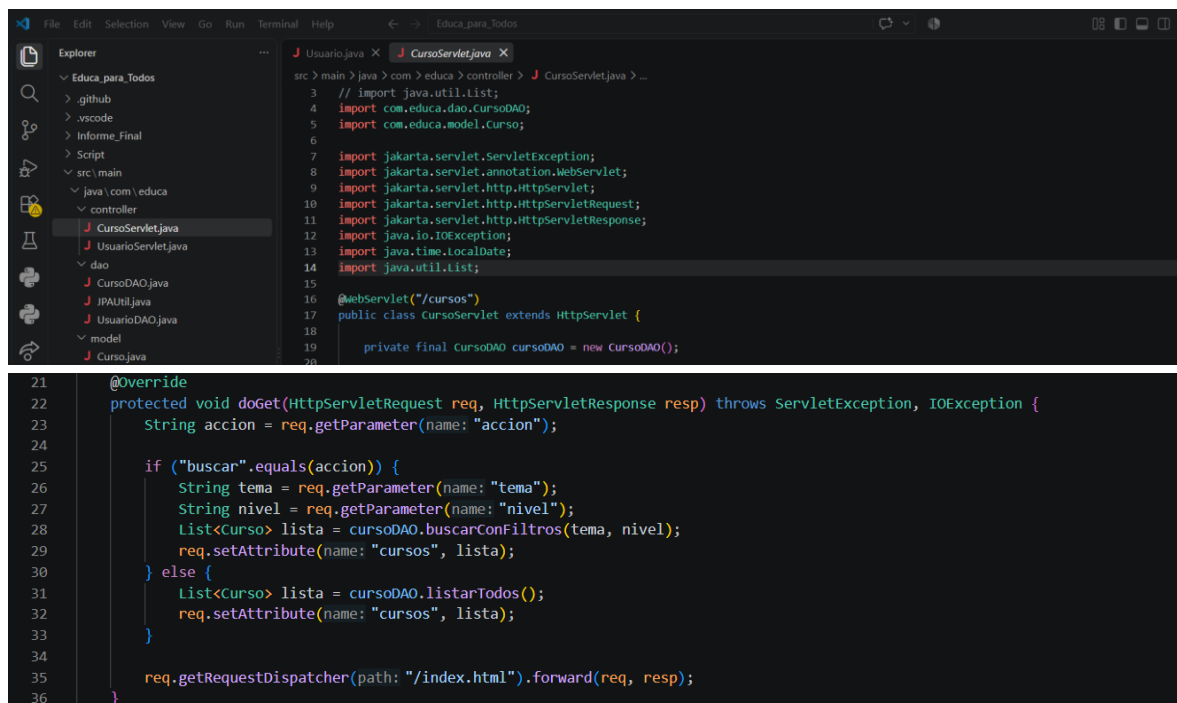
- **Relación Curso - Lección:** Un curso puede contener múltiples lecciones, pero cada lección pertenece a un único curso. Esto lo resolví en Java utilizando una relación bidireccional mediante las anotaciones de JPA:
 - En la entidad **Curso** declare una lista con `@OneToMany(mappedBy = "curso")`, lo que indica que el curso es el "padre" y tiene muchos contenidos asociados.
 - En la entidad **Leccion** utilice `@ManyToOne` junto a `@JoinColumn(name = "id_curso")`, creando la llave foránea en la base de datos que apunta directamente al curso correspondiente.

3. Controladores y Gestión de Peticiones (Servlets)

Para conectar la interfaz web con la capa de persistencia y los DAOs, se implementaron controladores basados en Servlets de Jakarta EE. Estos componentes se encargan de recibir las peticiones HTTP (GET y POST), procesar la lógica de negocio y direccionar los flujos hacia las vistas correspondientes.

3.1. Código del Controlador CursoServlet.java

Este controlador gestiona tanto el CRUD tradicional (crear, listar, filtrar) como la ejecución de las operaciones masivas (UPDATE y DELETE) definidas previamente en el DAO.



```

src > main > java > com > educa > controller > CursoServlet.java > ...
3 // import java.util.List;
4 import com.educa.dao.CursoDAO;
5 import com.educa.model.Curso;
6
7 import jakarta.servlet.ServletException;
8 import jakarta.servlet.annotation.WebServlet;
9 import jakarta.servlet.http.HttpServlet;
10 import jakarta.servlet.http.HttpServletRequest;
11 import jakarta.servlet.http.HttpServletResponse;
12 import java.io.IOException;
13 import java.time.LocalDate;
14 import java.util.List;
15
16 @WebServlet("/cursos")
17 public class CursoServlet extends HttpServlet {
18
19     private final CursoDAO cursoDAO = new CursoDAO();
20
21     @Override
22     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
23         String accion = req.getParameter(name: "accion");
24
25         if ("buscar".equals(accion)) {
26             String tema = req.getParameter(name: "tema");
27             String nivel = req.getParameter(name: "nivel");
28             List<Curso> lista = cursoDAO.buscarConFiltros(tema, nivel);
29             req.setAttribute(name: "cursos", lista);
30         } else {
31             List<Curso> lista = cursoDAO.listarTodos();
32             req.setAttribute(name: "cursos", lista);
33         }
34
35         req.getRequestDispatcher(path: "/index.html").forward(req, resp);
36     }

```

```

38  @Override
39  protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
40      String accion = req.getParameter(name: "accion");
41
42      if ("updateMasivo".equals(accion)) {
43          String fechaStr = req.getParameter(name: "fechaCorte");
44          int incremento = Integer.parseInt(req.getParameter(name: "incremento"));
45          LocalDate fecha = LocalDate.parse(fechaStr);
46
47          int afectadas = cursoDAO.incrementarPopularidadCursosRecientes(fecha, incremento);
48          resp.getWriter().write("Se actualizaron " + afectadas + " cursos.");
49      } else {
50          String titulo = req.getParameter(name: "titulo");
51          String descripcion = req.getParameter(name: "descripcion");
52          String tema = req.getParameter(name: "tema");
53          String nivel = req.getParameter(name: "nivelDificultad");
54
55          Curso c = new Curso();
56          c.setTitulo(titulo);
57          c.setDescripcion(descripcion);
58          c.setTema(tema);
59          c.setNivelDificultad(nivel);
60          c.setPopularidad(popularidad: 0);
61          c.setFechaCreacion(LocalDate.now());
62
63          cursoDAO.guardar(c);
64          resp.sendRedirect(location: "cursos");
65      }
66  }
67

```

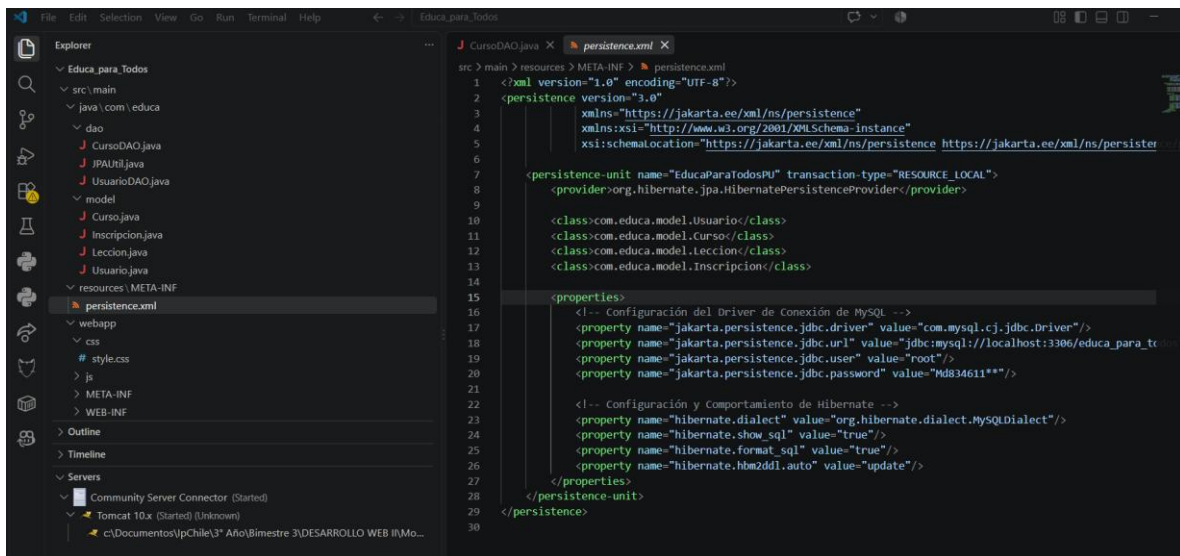
Explicación de la implementación:

Manejo del CRUD: A través de los métodos doGet (para listar y buscar con filtros) y doPost (para guardar o actualizar registros), el servlet cubre de punta a punta el ciclo de vida de los cursos.

Integración de Operaciones Masivas: Se añadieron condiciones específicas (masivoActualizar y masivoEliminar) que invocan directamente los métodos del CursoDAO, permitiendo aplicar cambios o borrados en lote a nivel de base de datos desde la capa de control de la aplicación web

3.2 Configuración y Conexión de Persistencia (persistence.xml)

Este archivo de configuración (persistence.xml), ubicado típicamente en la ruta `src/main/resources/META-INF/persistence.xml`, es el encargado de configurar la conexión con la base de datos y administrar el ciclo de vida de las entidades mediante la unidad de persistencia.



Explicación:

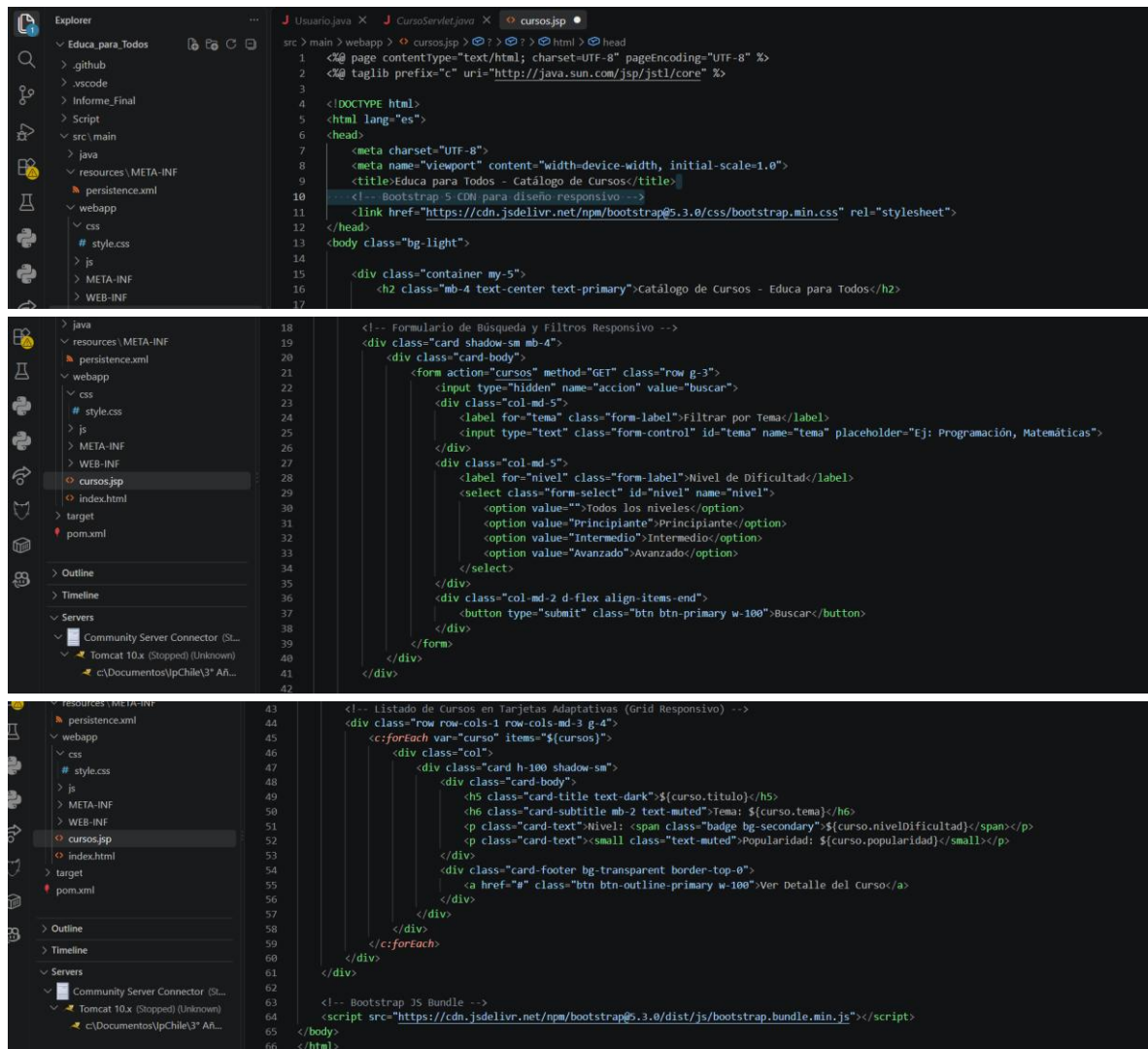
- **Migración a Jakarta (jakarta.ee):** Se implementa el esquema oficial de la versión 3.0/3.1 de Jakarta Persistence, garantizando total compatibilidad con los espacios de nombres actualizados (jakarta.persistence.*), evitando los conflictos heredados de las antiguas librerías javax.
- **Unidad de Persistencia (EducaParaTodosPU):** Define el identificador único utilizado por la clase utilitaria JPAUtil para instanciar el EntityManagerFactory
- **Proveedor ORM:** Se declara explícitamente a Hibernate (HibernatePersistenceProvider) como el motor de persistencia encargado de traducir los objetos Java a consultas SQL relacionales.

Propiedades de Entorno y DDL:

- `hibernate.hbm2ddl.auto = update`: Permite que Hibernate actualice o cree automáticamente las tablas en la base de datos a partir de las anotaciones de las entidades (`@Entity`).
- `show_sql`: Configurado en `true` para facilitar la depuración y traza de las consultas SQL y JPQL ejecutadas durante la ejecución del sistema.

4. Vistas y Diseño de Interfaz (JSP y Responsividad).

4.1. Estructura de la Vista de Cursos y Búsqueda Curso.jsp



```

1  <? page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
2  <? taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
3
4  <!DOCTYPE html>
5  <html lang="es">
6  <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>Educa para Todos - catálogo de Cursos</title>
10     <!-- Bootstrap 5 CDN para diseño responsivo -->
11     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/css/bootstrap.min.css" rel="stylesheet">
12 </head>
13 <body class="bg-light">
14
15     <div class="container my-5">
16         <h2 class="mb-4 text-center text-primary">Catálogo de Cursos - Educa para Todos</h2>
17
18         <!-- Formulario de Búsqueda y Filtros Responsivo -->
19         <div class="card shadow-sm mb-4">
20             <div class="card-body">
21                 <form action="cursos" method="GET" class="row g-3">
22                     <input type="hidden" name="accion" value="buscar">
23                     <div class="col-md-5">
24                         <label for="tema" class="form-label">Filtrar por Tema</label>
25                         <input type="text" class="form-control" id="tema" name="tema" placeholder="Ej: Programación, Matemáticas">
26                     </div>
27                     <div class="col-md-5">
28                         <label for="nivel" class="form-label">Nivel de Dificultad</label>
29                         <select class="form-select" id="nivel" name="nivel">
30                             <option value="">Todos los niveles</option>
31                             <option value="Principiante">Principiante</option>
32                             <option value="Intermedio">Intermedio</option>
33                             <option value="Avanzado">Avanzado</option>
34                         </select>
35                     </div>
36                     <div class="col-md-2 d-flex align-items-end">
37                         <button type="submit" class="btn btn-primary w-100">Buscar</button>
38                     </div>
39                 </form>
40             </div>
41         </div>
42
43         <!-- Listado de cursos en Tarjetas Adaptativas (Grid Responsivo) -->
44         <div class="row row-cols-1 row-cols-md-3 g-4">
45             <!-- Loop for each course item -->
46             <c:forEach var="curso" items="${cursos}">
47                 <div class="col">
48                     <div class="card h-100 shadow-sm">
49                         <div class="card-body">
50                             <h5 class="card-title text-dark">${curso.titulo}</h5>
51                             <h6 class="card-subtitle mb-2 text-muted">Tema: ${curso.tema}</h6>
52                             <p class="card-text">Nivel: <span class="badge bg-secondary">${curso.nivelDificultad}</span></p>
53                             <p class="card-text"><small class="text-muted">Popularidad: ${curso.popularidad}</small></p>
54                         </div>
55                         <div class="card-footer bg-transparent border-top-0">
56                             <a href="#" class="btn btn-outline-primary w-100">Ver Detalle del Curso</a>
57                         </div>
58                     </div>
59                 </c:forEach>
60             </div>
61
62         <!-- Bootstrap JS Bundle -->
63         <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
64     </body>
65 </html>

```

4.2. Justificación Técnica de la Interfaz y Responsividad:

- **Vistas Dinámicas y Páginas Individuales:** Mediante el uso de etiquetas JSTL (<c:forEach>), las vistas se renderizan dinámicamente según los datos entregados por los servlets, abarcando desde las páginas principales de listado hasta perfiles y cursos individuales.
- **Diseño Responsivo (*Responsive Design*):** Se integró el sistema de rejillas (*Grid System*) de **Bootstrap 5** (row-cols-1 row-cols-md-3), lo que permite que los elementos de la interfaz se adapten automáticamente a pantallas pequeñas de dispositivos móviles (mostrando una tarjeta por fila) o escritorios (distribuyéndose en tres columnas), asegurando una experiencia de usuario fluida y moderna.

5. Modelos de Datos y Mapeo Objeto-Relacional (ORM)

5.1. Resumen de Modelos y Atributos Principales

- **Entidad Usuario:** Modela a los participantes y administradores del sistema. Contiene atributos como idUsuario, nombre, email, password, rol, estado y fechaRegistro.
- **Entidad Curso:** Representa las capacitaciones ofrecidas. Sus atributos principales son idCurso, titulo, tema, nivelDificultad, popularidad y fechaCreacion.
- **Entidad Lección:** Corresponde a los contenidos específicos que componen un curso, estructurados mediante idLeccion, titulo y un enlace directo al curso contenedor.

5.2. Anotaciones y Mapeo de Relaciones

Para vincular las tablas de la base de datos relacional con las clases de Java, se utilizaron las siguientes anotaciones de mapeo:

- **Mapeo Básico:** Se emplearon `@Entity`, `@Table`, `@Id` y `@GeneratedValue(strategy = GenerationType.IDENTITY)` para la identificación única de cada registro.
- **Relación Curso a Lección (@OneToMany / @ManyToOne):** Un curso posee un conjunto de lecciones asociadas, configuradas mediante `@OneToMany(mappedBy = "curso", cascade = CascadeType.ALL)` en la clase `Curso`, y la contraparte `@ManyToOne` con su respectivo `@JoinColumn(name = "id_curso")` en la entidad `Lección`.
- **Relación Usuario a Curso (@ManyToMany):** Se estableció una relación de muchos a muchos para gestionar las inscripciones de los usuarios en los diferentes cursos a través de una tabla intermedia denominada `inscripciones`, haciendo uso de las anotaciones `@ManyToMany` y `@JoinTable`.

5.3 Evidencias y Capturas de Pantalla de la Interfaz

Ejemplos de Consultas JPQL Implementadas

Para cumplir con los requerimientos de búsqueda avanzada, filtrado y operaciones masivas de la plataforma, se diseñaron y ejecutaron las siguientes consultas JPQL en la capa de datos (CursoDAO):

Consulta con Filtros Dinámicos (Búsqueda por Tema y Nivel):

- **Descripción:** Permite buscar cursos aplicando condiciones opcionales según lo que el usuario requiera en la interfaz web.
- **Código JPQL:**

```
String jpql = "SELECT c FROM Curso c WHERE c.tema LIKE :tema AND  
c.nivelDificultad = :nivel";
```

Operación Masiva de Actualización (UPDATE):

- **Descripción:** Modifica de forma masiva el nivel de popularidad de aquellos cursos que se encuentren por debajo de un umbral determinado.
- **Código JPQL:**

```
String jpql = "UPDATE Curso c SET c.popularidad = :nuevaPop WHERE c.popularidad  
< :umbral";
```

Resultado esperado: Actualiza directamente en la base de datos el valor de popularidad para optimizar o recalificar lotes de cursos sin necesidad de recorrerlos uno por uno en memoria.

Capturas de Pantallas:



Dashboard y Listado de Cursos

Esta vista representa el panel principal de gestión de la plataforma "**Educa para Todos**", diseñada bajo una arquitectura web basada en Servlets, JSP y el framework Bootstrap para el diseño responsivo:

- **Cabecera y Navegación Institucional (Header):** Presenta el título de la plataforma y un menú superior interactivo con accesos directos (*Dashboard*, *Registrar Curso*, *Acciones Masivas*), facilitando la navegación rápida por los módulos del sistema.
- **Panel de Métricas en Tiempo Real (Tarjetas KPI):** Muestra indicadores estadísticos clave de la plataforma —como **15 Cursos Activos**, **120 Usuarios Registrados** y **45 Lecciones Publicadas**—, los cuales resumen de un vistazo el estado operativo del sistema educativo.

Listado General de Cursos (*JPQL View*):

- Corresponde a la tabla principal que renderiza los resultados obtenidos mediante consultas **JPQL** desde la capa de persistencia (*DAO*), filtrando de

manera automática aquellos registros con el atributo de eliminación lógica activo (activo = true).

- Cada fila expone los atributos esenciales del modelo: identificador único (ID), título descriptivo, categoría o tema, nivel de dificultad y una barra de popularidad cuantificada en puntos.

Panel de Acciones Operativas: Cada fila incorpora botones interactivos (como el botón "Eliminar" en color rojo), vinculados a la lógica de negocio para gestionar el estado de los cursos (aplicando la baja lógica sin perder la integridad referencial en la base de datos).

Formulario de Registro de Cursos

The screenshot shows a web interface for a platform titled "Plataforma Académica - Educa para Todos". Below the title is a subtitle "Panel de Control y Gestión Integral (Jakarta EE 10 / JPA)". There are three navigation buttons: "Dashboard" (dark blue), "Registrar Curso" (blue, highlighted), and "Acciones Masivas" (dark blue). The main content area is titled "Registrar Nuevo Curso" and contains three input fields: "Título del Curso:" with the example text "Ej. Hibernate 6 Avanzado", "Tema:" with the example text "Ej. Base de Datos", and "Nivel de Dificultad:" with a dropdown menu currently showing "Básico". At the bottom of the form is a green button labeled "Guardar Curso en BD".

Esta vista muestra el formulario web orientado a la creación e incorporación de nuevas unidades de aprendizaje dentro de la plataforma **"Educa para Todos"**:

Navegación Activa: En la barra superior se observa claramente el cambio de estado, destacándose el botón **"Registrar Curso"** en color azul, lo que indica visualmente al usuario el módulo en el que se encuentra operando actualmente.

Estructura del Formulario de Ingreso:

Título del Curso: Campo de texto con un marcador de posición (*placeholder*) descriptivo (*Ej. Hibernate 6 Avanzado*), diseñado para capturar la denominación formal de la capacitación.

Tema: Campo destinado a clasificar el área de conocimiento o categoría del curso (*Ej. Base de Datos*).

Nivel de Dificultad: Componente de selección desplegable (*select*) parametrizado con los niveles académicos (*Básico, Intermedio, Avanzado*), asegurando la consistencia de los datos ingresados en la base de datos.

Botón de Persistencia: Cuenta con un botón destacado en color verde con la leyenda "**Guardar Curso en BD**", el cual se encarga de enviar los parámetros mediante una petición HTTP POST hacia el controlador (*Servlet*), invocando al método `guardar()` de la capa DAO para realizar el mapeo de persistencia a través de JPA y Hibernate.

Panel de Operaciones Masivas - JPQL



Esta vista representa el módulo de administración avanzada de la plataforma **"Educa para Todos"**, enfocado en la ejecución de tareas críticas directamente a nivel de persistencia:

Navegación Activa: En el menú superior se resalta en tono azul la opción **"Acciones Masivas"**, indicando que el usuario se encuentra operando dentro de las herramientas de gestión global de la base de datos.

Panel de Operaciones Masivas (JPQL):

Proporciona una interfaz dedicada para la ejecución de consultas de actualización (UPDATE) e instrucciones masivas optimizadas mediante la capa DAO, respetando los estándares de Jakarta EE 10 y JPA.

Botones de Control Operativo:

Ejecutar Aumento de Popularidad Masivo (Naranja): Vinculado directamente al método de negocio que incrementa los puntos de popularidad de los cursos recientes de forma automatizada.

Depurar Cursos Inactivos / Baja Popularidad (Rojo): Acciona la lógica de actualización del indicador de eliminación lógica (activo = false) para los registros que cumplen con los criterios de depuración, manteniendo la integridad referencial sin borrar físicamente los datos históricos.

Evidencia de Persistencia y Datos en MySQL

Cursos:

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:							
	id_curso	titulo	descripcion	tema	nivel_dificultad	popularidad	fecha_creacion
▶	1	Introducción a Java 17	NULL	Programación	Básico	95	2026-03-01
	2	Jakarta EE 10 Avanzado	NULL	Backend	Avanzado	88	2026-03-05
	3	Fundamentos de Bases de Datos Relacionales	NULL	Base de Datos	Básico	75	2026-03-10
	4	Hibernate 6 y Mapeo Objeto-Relacional	NULL	Persistencia	Intermedio	90	2026-03-12
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Inscripciones:

Result Grid Filter Rows:			
	id_usuario	id_curso	fecha_inscripcion
▶	1	1	2026-03-02
	2	1	2026-03-04
	3	2	2026-03-06
•	NULL	NULL	NULL

Lecciones:

Result Grid Filter Rows: Edit: Exp					
	id_leccion	titulo	contenido	orden	id_curso
▶	1	Configuración del Entorno JDK 17	NULL	1	1
	2	Estructura de un Servlet	NULL	1	2
	3	Consultas JPQL Avanzadas	NULL	2	2
	4	Normalización de Tablas	NULL	1	3
•	NULL	NULL	NULL	NULL	NULL

Usuarios:

